Synthetic Neuronal Network Topology Generation: Distance-Dependent Sparse Connectivity with Dale's Law

Jan Balewski, NERSC

March 23, 2026

Abstract

We describe the pipeline implemented in `gen_daleMatrices4.py`: (1) build a spatially embedded, Dale-law-compliant sparse network A_{true} with target spectral radius $R < 1$ and a bias vector B; (2) simulate spike counts under a Poisson GLM, either **Model A** (lag-1, no off-diagonal memory) or **Model B** (lag-M: diagonal lag-1 drive plus an off-diagonal history filtered by a damped-oscillator kernel κ_ℓ). A third variant (Model C: additional diagonal adaptation kernel) is discussed separately but is *not* implemented in this script.

Contents

1 Overview of the Method

The objective is to produce synthetic spike-count time series $Y_t \in \mathbb{Z}_{\geq 0}^N$ ($t = 1, \ldots, T$) with configurable spatial connectivity and Poisson GLM dynamics.

Stage 1 — Network construction (Section 2). N neurons are sampled from a regular grid on $[0, L] \times [0, H]$ and connected by distance-dependent out-degree sampling with Dale's law. Outputs: A_{true} with $\rho(A_{\text{true}}) = R$ and bias B.

Stage 2 — Spike generation (Section 3). **Model A:** lag-1 recurrence on all couplings. **Model B:** diagonal term uses Y_{t-1} only; off-diagonal block uses lags $1, \ldots, M$ with kernel κ_ℓ (Section 3.2). The number of lags M is computed from the oscillation period (not a separate CLI flag). Model C (Section 3.3) is not implemented in code.

Notation conventions. A_{true} denotes the connectivity matrix throughout. $A_{\text{true}}^{\text{diag}} = \text{diag}((A_{\text{true}})_{11}, \ldots, (A_{\text{true}})_{NN})$ is its diagonal and $A_{\text{true}}^{\text{off}} = A_{\text{true}} - A_{\text{true}}^{\text{diag}}$ its off-diagonal part. In Model B (Section 3.2), $A^{\text{diag}} \in \mathbb{R}^N$ denotes the *vector* of diagonal entries $(A_{\text{true}})_{ii}$, parallel to the matrix shorthand A^{off}. $Y_t \in \mathbb{Z}_{\geq 0}^N$ is the vector of spike counts in time bin t, where each bin has width Δt seconds. $B \in \mathbb{R}^N$ is the bias (log-baseline-rate) vector. $\eta_{\text{clip}} = 5$ is a saturation threshold that prevents numerical overflow in the exponential link function. Column convention: $(A_{\text{true}})_{ij}$ is the weight from presynaptic neuron j to postsynaptic neuron i.

2 Generation of A_{true} and B

2.1 Parameters

The script takes (among others): N and N_{exc} (default $0.8\,N$); rectangular domain $[0, L] \times [0, H]$; grid spacing $d_{\min} > 0$ (`--placement_min_dist`); distance-kernel exponent $\delta_{\text{ker}} \in \{1, 2\}$ (third entry of `--placement_H_L_delta`); out-degree range from two fractions $p_{\text{lo}} < p_{\text{hi}}$ via $k_{\min} = \max(1, \lfloor p_{\text{lo}} N \rfloor)$, $k_{\max} = \min(N - 1, \lfloor p_{\text{hi}} N \rfloor)$ (defaults 0.05, 0.2); weight spread $v \in (0, 1)$; target spectral radius $R \in (0, 1)$.

Variables. $\mathbf{P} \in \mathbb{R}^{N \times 2}$: neuron spatial positions, row $i = (x_i, y_i)$, sorted by (x_i, y_i) lexicographically after placement. $\mathbf{D} \in \mathbb{R}^{N \times N}$: pairwise Euclidean distances. $\mathbf{V} \in \mathbb{R}^{N \times N}$: distance-kernel affinity matrix. $\mathbf{E} \in \{-1, 0, +1\}^{N \times N}$: signed topology matrix; $E_{ij} \neq 0$ means neuron j drives neuron i. $\tau_j \in \{E, I\}$: type of neuron j; $\sigma_j = +1$ (exc) or -1 (inh). k_j: out-degree of neuron j. $r = N_{\text{exc}}/N_{\text{inh}} = 4$: E-I ratio.

2.2 Algorithm

Algorithm 1 Synthetic Neuronal Network Topology

1: **Phase 1:** Construct placement grid; sample N nodes; assign types E/I and signs σ_j
2: **Phase 2:** Compute distance matrix $\mathbf{D}$ and affinity matrix $\mathbf{V}$
3: **Phase 3:** Sample sparse signed topology $\mathbf{E}$
4: **Phase 4:** Assign edge weights; enforce E-I ratio and rescale to spectral radius R
5: **Phase 5:** Generate bias vector B

2.2.1 Phase 1: Grid-Based Neuron Placement and Type Assignment

Step 1.1 — Construct placement grid. Tile $[0, L] \times [0, H]$ with spacing $d_{\min}$:

$$\mathcal{G} = \{(m\, d_{\min},\ n\, d_{\min}) :\ m = 0, \ldots, \lfloor L/d_{\min} \rfloor, \quad n = 0, \ldots, \lfloor H/d_{\min} \rfloor\}. \tag{1}$$

$$N_{\mathcal{G}} = \big(\lfloor L/d_{\min} \rfloor + 1\big)\big(\lfloor H/d_{\min} \rfloor + 1\big). \tag{2}$$

A valid configuration requires $N_{\mathcal{G}} \geq N$; if not satisfied, $d_{\min}$ must be reduced or L increased.

Step 1.2 — Sample neuron positions. Draw N nodes uniformly at random *without replacement* from $\mathcal{G}$ and discard the remaining $N_{\mathcal{G}} - N$ nodes. Store accepted positions as rows of $\mathbf{P}$, sorted by (x, y) (lexicographic).

Step 1.3 — Assign neuron types. Sample N_{exc} neuron indices without replacement as excitatory; the remainder are inhibitory. Define:

$$\sigma_j = \begin{cases} +1 & \tau_j = E, \\ -1 & \tau_j = I. \end{cases} \tag{3}$$

2.2.2 Phase 2: Distance D and Affinity Matrix V

Pairwise distances use Euclidean geometry; for the affinity, the code sets $D_{ii} = \infty$ on the diagonal so self-entries are excluded before

$$V_{ij} = D_{ij}^{-\delta_{\text{ker}}}, \quad i \neq j; \qquad V_{ii} = 0. \tag{4}$$

The saved distance matrix in `.simTruth.npz` uses zeros on the diagonal. $\delta_{\text{ker}} \in \{1, 2\}$ controls decay steepness; sampled grid nodes satisfy spacing $\geq d_{\min}$ so pairwise distances among distinct neurons need not hit 0.

2.2.3 Phase 3: Sparse Topology Sampling

For each presynaptic neuron $j = 1, \dots, N$ independently:

1. Sample out-degree $k_j \sim \text{Uniform\_int}(k_{\min}, k_{\max})$.
2. Normalize affinity over valid targets:

$$p_{ij} = \frac{V_{ij}}{\sum_{i' \neq j} V_{i'j}}, \qquad i \neq j. \tag{5}$$

3. Draw k_j distinct postsynaptic targets $\mathcal{T}_j \subset \{1, \dots, N\} \setminus \{j\}$ without replacement according to $\{p_{ij}\}$.

Build the signed topology matrix E with negative self-loops:

$$E_{ij} = \begin{cases} \sigma_j & i \in \mathcal{T}_j,\ i \neq j, \\ -1 & i = j, \\ 0 & \text{otherwise.} \end{cases} \tag{6}$$

2.2.4 Phase 4: Weight Assignment and Spectral Normalization

Raw weights. Draw unsigned weights magnitudes $U_{ij} \sim \text{Uniform}(1 - v, 1 + v)$ independently for each non-zero entry of $\mathbf{E}$, then scale by neuron type:

$$W_{ij} = \begin{cases} +U_{ij} & E_{ij} = +1, \\ -r\, U_{ij} & E_{ij} = -1,\ i \neq j, \\ -U_{ii} & i = j, \\ 0 & E_{ij} = 0. \end{cases} \tag{7}$$

Spectral normalization. A single global rescaling sets $\rho(A_{\text{true}})$ to exactly R:

$$\rho_0 = \max_k |\lambda_k(W)|, \qquad A_{\text{true}} \leftarrow \frac{R}{\rho_0} W, \tag{8}$$

where $\lambda_k(W)$ are the (possibly complex) eigenvalues of W. Assume $\rho_0 > 0$. Rescaling preserves all signs and the zero pattern, so Dale's law, E-I ratio, and negative self-loops are all maintained. For $R < 1$, contractivity of A_{true} is a sufficient condition for stationarity of the Poisson GLM.

2.2.5 Phase 5: Bias Vector Generation

Let $f_{\text{idle}} \in [f_{\min}, f_{\max}]$ be a *target idle firing-rate band* in Hz (argument `idleRate`, default $[15, 30]$). Independent draws

$$B_i \sim \text{Uniform}\big(\ln(Rf_{\min}), \ln(Rf_{\max})\big), \tag{9}$$

then all excitatory indices receive a fixed downward shift $B_i \leftarrow B_i - 2$ (hard-coded in the script). Inhibitory components are unchanged. This matches `set_flat_selfSpiking` in code.

2.3 Output Summary

Array	Shape	Type	Description
P	$N \times 2$	float	Neuron positions (x_i, y_i), sorted by (x, y)
D	$N \times N$	float	Pairwise distances; diagonal 0 in output
τ	N	int	Type labels: 0=exc, 1=inh
σ	N	± 1	Synaptic signs
E	$N \times N$	$\{-1, 0, +1\}$	Signed topology (Eq. 6)
k	N	int	Out-degree, $k_j \in [k_{\min}, k_{\max}]$
A_{true}	$N \times N$	float	Weighted connectivity, $\rho(A_{\text{true}}) = R$
B	N	float	Bias vector (Eq. 9)

Table 1: Output arrays of Stage 1. A_{true} and B are passed to the spike generator (Section 3).

3 Generation of Spike Trains

Given A_{true}, B, time-bin width Δt, horizon T, and $\eta_{\text{clip}} = 5$ (hard-coded), Models A and B are implemented. Bin t indexes rows Y_t (code uses $t = 0, \ldots, T-1$). **Model A:** $Y_0 \sim \text{Poisson}(\exp(B)\Delta t)$, then Eq. (10) for $t \geq 1$. **Model B:** the first M rows are set to zero (pre-history), with $M =$ `mem_lag_steps` computed from τ_{mem} (Section 3.2); dynamics follow that section for $t \geq 1$.

3.1 Model A: Lag-1, No Memory, Not Bursting

The simplest model conditions only on the immediately preceding bin:

$$\eta_t = A_{\text{true}} Y_{t-1} + B, \qquad \tilde{\eta}_t = \min(\eta_t,\ \eta_{\text{clip}}), \qquad \mu_t = \exp(\tilde{\eta}_t)\,\Delta t, \qquad Y_t \sim \text{Poisson}(\mu_t). \tag{10}$$

Here $\min(\cdot)$ and $\exp(\cdot)$ act element-wise. Because $\rho(A_{\text{true}}) < 1$, the process admits a stationary distribution.

3.2 Model B: Lag-M, Single Off-Diagonal Kernel, Bursting

3.2.1 Off-diagonal kernel κ_ℓ (damped oscillator)

Let $\tau_{\text{mem}} > 0$ be the oscillation period in **seconds** (required; the code asserts $\tau_{\text{mem}} > 0$). Superscript $^{(s)}$ marks quantities expressed in **discrete steps** (time bins). Define $\tau^{(s)} = \text{round}(\tau_{\text{mem}}/\Delta t) \geq 1$, the oscillation period in bins. Let $Q > 0$ (quality factor); the CLI passes `mem_Q` and `mem_tau` (`--time_kernel_q_tau`). Set $\omega = 2\pi/\tau^{(s)}$ and $\gamma = \pi/(Q\,\tau^{(s)})$. The off-diagonal history uses only lags $\ell = 1, \ldots, M$ with

$$M = \max\Big(1,\ \lfloor 3\tau^{(s)}/4 \rfloor\Big) \tag{11}$$

This choice sets the *truncation depth* $M \approx \frac{3}{4}\tau^{(s)}$, so $\omega M \approx \frac{3\pi}{2}$ (about 1.5π radians of oscillation phase across the retained lags). For $\ell = 1, \ldots, M$, define raw weights

$$\tilde{\kappa}_\ell = \exp(-\gamma\ell)\,\cos(\omega\ell). \tag{12}$$

There is *no* $\ell = 0$ term. The implementation then fixes the scale by normalizing to the first lag: $\kappa_\ell = \tilde{\kappa}_\ell/\tilde{\kappa}_1 = 1.0$

3.2.2 Evolution

Write $A^{\text{diag}} \in \mathbb{R}^N$ with $(A^{\text{diag}})_i = (A_{\text{true}})_{ii}$, and let $A^{\text{off}} = A_{\text{true}}$ with zero diagonal. The off-diagonal spike-history vector is $S_t = \sum_{\ell=1}^{M} \kappa_\ell\, Y_{t-\ell}$ (vector sum). For $t \geq 1$,

$$\eta_t = A^{\text{diag}} \odot Y_{t-1} + A^{\text{off}} S_t + B, \tag{13}$$

then $\tilde{\eta}_t = \min(\eta_t, \eta_{\text{clip}})$, $\mu_t = \exp(\tilde{\eta}_t)\,\Delta t$, and $Y_t \sim \text{Poisson}(\mu_t)$ element-wise as in Eq. (10). The first M rows $Y_0, \ldots, Y_{M-1}$ are held at 0 (pre-history).

Relation to Model A. Model A is a separate code path (`--spike_model A`); it is **not** recovered by formally setting $M = 1$ in Model B.

3.2.3 Stability

`gen_daleMatrices4.py` does not impose an extra analytic stability test for Model B beyond rescaling A_{true} to spectral radius $R < 1$ (Eq. 8). A sufficient condition for a two-kernel extension appears in Section 3.3 (Model C).

3.3 Model C: Lag-M, Two Kernels, Bursting - not implemented

3.3.1 Adaptation kernel for the diagonal

Spike-frequency adaptation is modeled as a purely inhibitory, exponentially decaying self-history kernel with time constant τ_{adapt} (in time bins) and amplitude $\alpha_{\text{adapt}} > 0$. Let M_{d} denote the number of lags retained for the diagonal kernel (which may differ from M). The unnormalized weights are:

$$\tilde{h}_\ell^{\text{diag}} = \exp\left(-\frac{\ell}{\tau_{\text{adapt}}}\right), \qquad \ell = 1, \ldots, M_{\text{d}}. \tag{14}$$

Normalizing and attaching the inhibitory sign:

$$h_\ell^{\text{diag}} = -\,\alpha_{\text{adapt}}\,\frac{\tilde{h}_\ell^{\text{diag}}}{\sum_{k=1}^{M_{\text{d}}} \tilde{h}_k^{\text{diag}}}, \qquad \ell = 1, \ldots, M_{\text{d}}. \tag{15}$$

All weights are negative, ensuring that a neuron's own past activity is always suppressive. The dimensionless amplitude α_{adapt} controls the overall strength of adaptation relative to the diagonal entries of A_{true}.

3.3.2 Evolution equations

The history drive now has two kernel-filtered components:

$$H_t = A_{\text{true}}^{\text{diag}} \left(\sum_{\ell=1}^{M_\text{d}} h_\ell^{\text{diag}} \, Y_{t-\ell} \right) + A_{\text{true}}^{\text{off}} \left(\sum_{\ell=1}^{M} h_\ell^{\text{off}} \, Y_{t-\ell} \right). \tag{16}$$

Note that, unlike Model B, the diagonal term now integrates over M_d lags rather than using only Y_{t-1}. Spike generation proceeds identically:

$$\eta_t = H_t + B, \qquad \tilde{\eta}_t = \min(\eta_t,\ \eta_{\text{clip}}), \qquad \mu_t = \exp(\tilde{\eta}_t)\,\Delta t, \qquad Y_t \sim \text{Poisson}(\mu_t). \tag{17}$$

3.3.3 Reduction to Model B

Setting $M_\text{d} = 1$ and $\alpha_{\text{adapt}} = 1$ gives $h_1^{\text{diag}} = -1$, so the diagonal term becomes $-A_{\text{true}}^{\text{diag}} Y_{t-1}$. Since $(A_{\text{true}})_{ii} < 0$ by construction (negative self-loops), the product $-A_{\text{true}}^{\text{diag}} Y_{t-1} = |A_{\text{true}}^{\text{diag}}| Y_{t-1}$ has the wrong sign. To recover Model B exactly, one must instead set $\alpha_{\text{adapt}} = -1$ (i.e. disable the adaptation sign flip) or, equivalently, simply bypass the diagonal kernel and use $A_{\text{true}}^{\text{diag}} Y_{t-1}$ directly. In practice, Model C is not intended to reduce to Model B; rather, Model B is the appropriate choice when adaptation is not modeled.

3.3.4 Stability

A sufficient condition for stability of Model C is:

$$\rho\!\left(A_{\text{true}}^{\text{off}}\right) \cdot \sum_{\ell=1}^{M} |h_\ell^{\text{off}}| \; + \; \max_i \big|(A_{\text{true}})_{ii}\big| \cdot \sum_{\ell=1}^{M_\text{d}} |h_\ell^{\text{diag}}| \; < \; 1. \tag{18}$$

The first term equals $\rho(A_{\text{true}}^{\text{off}})$ by the off-diagonal kernel normalization. The second term equals $\alpha_{\text{adapt}} \max_i |(A_{\text{true}})_{ii}|$ by the diagonal kernel normalization. Both contributions must be kept small enough that their sum remains below unity.

3.4 Parameter Choices

The script defaults include $\Delta t = 0.01\,\text{s}$ and `--time_kernel_q_tau` $= (Q, \tau_{\text{mem}}) = (3, 0.4)$ in code (two floats: `mem_Q`, `mem_tau` in seconds). The lag count $M =$ `mem_lag_steps` equals the length of `_offdiag_time_kernel` (computed from $\tau^{(s)}$ as above), not passed separately. For typical simulations choose $T =$ `num_steps` well above M so the trajectory spends most bins outside the initial zero pre-history.

Model C-specific parameter guidance remains in Section 3.3.

3.5 Notation Summary

For reference, all symbols used in the spike generation models are collected here.

Symbol	Domain	Description
N	$\mathbb{Z}_{>0}$	Number of neurons
T	$\mathbb{Z}_{>0}$	Number of time bins
Δt	$\mathbb{R}_{>0}$	Time-bin width (seconds); default 0.01 s
Y_t	$\mathbb{Z}_{\geq 0}^N$	Spike-count vector in bin t
A_{true}	$\mathbb{R}^{N\times N}$	Connectivity matrix from Section 2
$A_{\text{true}}^{\text{diag}}$	$\mathbb{R}^{N\times N}$	Diagonal of A_{true} (self-coupling)
$A_{\text{true}}^{\text{off}}$	$\mathbb{R}^{N\times N}$	Off-diagonal of A_{true} (cross-neuron coupling)
A^{diag}	$\mathbb{R}^N$	Vector $(A_{\text{true}})_{ii}$; Model B shorthand (Eq. 13)
B	$\mathbb{R}^N$	Bias vector (log-baseline rate)
η_{clip}	$\mathbb{R}_{>0}$	Clipping threshold; default 5
Off-diagonal kernel (Model B; Model C off-part)		
τ_{mem}	$\mathbb{R}_{>0}$	Oscillation period (**seconds**); stored as `mem_tau`; $\tau^{(s)} = \text{round}(\tau_{\text{mem}}/\Delta t)$
Q	$\mathbb{R}_{>0}$	Quality factor; stored as `mem_Q`
ω	$\mathbb{R}_{>0}$	$2\pi/\tau^{(s)}$
γ	$\mathbb{R}_{>0}$	$\pi/(Q\,\tau^{(s)})$
M	$\mathbb{Z}_{>0}$	`mem_lag_steps`$= \max(1, \lfloor 3\tau^{(s)}/4\rfloor)$; history lags $\ell = 1, \ldots, M$
κ_ℓ	$\mathbb{R}$	Eq. 12, $\kappa_\ell = \tilde{\kappa}_\ell/\tilde{\kappa}_1$
Diagonal adaptation kernel (Model C only)		
τ_{adapt}	$\mathbb{R}_{>0}$	Adaptation time constant (time bins); default 10
α_{adapt}	$\mathbb{R}_{>0}$	Adaptation amplitude; default 0.5
M_{d}	$\mathbb{Z}_{>0}$	Number of diagonal history lags
h_ℓ^{diag}	$\mathbb{R}_{\leq 0}$	Diagonal kernel weight at lag ℓ (Eq. 15)

Table 2: Complete notation for the spike generation models (Section 3). All symbols from Section 2 ($\mathbf{P}$, $\mathbf{E}$, σ, τ_j, r, R, δ_{ker}, etc.) retain their definitions from that section.

Appendix

.0.1 Excitable memory kernel

Experimental organoid recordings show brief synchronous bursts separated by long, irregular quiescent intervals. This pattern is characteristic of an excitable system rather than an oscillator: recent activity first promotes further firing (burst build-up) and then strongly suppresses it (post-burst refractoriness). We model this with a difference-of-exponentials kernel:

$$h(\tau) = e^{-\tau/\tau_{\text{exc}}} - \beta\, e^{-\tau/\tau_{\text{inh}}}, \qquad \tau > 0, \tag{19}$$

where $\tau_{\text{exc}} \ll \tau_{\text{inh}}$ and $\beta \in (0,1)$. The fast excitatory lobe (τ_{exc}, a few tens of milliseconds) creates the burst; the slow inhibitory lobe (τ_{inh}, several hundred milliseconds) enforces a prolonged refractory period. The kernel crosses zero at $\tau^* = \frac{\tau_{\text{exc}}\,\tau_{\text{inh}}}{\tau_{\text{inh}}-\tau_{\text{exc}}} \ln(1/\beta)$. Because the inhibitory tail decays slowly, the network remains quiescent until the suppression wears off and a noise fluctuation triggers the next burst, producing irregular inter-burst intervals without a fixed oscillation period.

Sampling at integer lags and normalizing gives the discrete weights:

$$\tilde{h}_\ell^{\text{off}} = \exp(-\ell/\tau_{\text{exc}}) - \beta \, \exp(-\ell/\tau_{\text{inh}}), \qquad h_\ell^{\text{off}} = \frac{\tilde{h}_\ell^{\text{off}}}{\sum_{k=1}^{M} |\tilde{h}_k^{\text{off}}|}, \qquad \ell = 1, \ldots, M. \tag{20}$$

A kernel support of $M \geq 3\tau_{\text{inh}}$ suffices to capture the exponential tail.

.0.2 Asymmetric damped-oscillator memory kernel

Experimental organoid recordings show brief synchronous bursts separated by long, irregular quiescent intervals. The standard damped-cosine kernel $h(\tau) = e^{-\gamma\tau}\cos(\omega\tau)$ produces symmetric positive and negative lobes, so the excitatory and suppressive phases of each cycle have equal duration. To obtain short bursts followed by prolonged silences we introduce a smooth, single-parameter frequency modulation that compresses the positive (excitatory) lobe and stretches the negative (suppressive) lobe.

Define the phase-modulated kernel

$$h(\tau) = e^{-\gamma\tau}\cos\!\big(\omega\tau + \delta\sin(\omega\tau)\big), \qquad \tau > 0, \tag{21}$$

with

$$\omega = \frac{2\pi}{\tau_{\text{mem}}}, \qquad \gamma = \frac{\pi}{Q\,\tau_{\text{mem}}}, \tag{22}$$

exactly as in the symmetric case. The new dimensionless parameter $\delta \in [0,1)$ controls the asymmetry between the two lobes.

The instantaneous frequency of the argument is $\dot{\Phi}(\tau) = \omega\big(1 + \delta\cos(\omega\tau)\big)$. Near the positive peak of the cosine ($\cos(\omega\tau) \approx +1$) the phase advances at rate $\omega(1+\delta)$, compressing the positive lobe; near the negative trough ($\cos(\omega\tau) \approx -1$) it advances at rate $\omega(1-\delta)$, stretching the negative lobe. The ratio of time spent in the negative versus positive half-cycle is approximately $(1+\delta)/(1-\delta)$. Setting $\delta = 0$ recovers the original symmetric kernel; $\delta = 0.6$ makes the negative lobe roughly four times longer than the positive one.

Sampling at integer lags and normalizing by the sum of absolute values gives the discrete weights:

$$\tilde{h}_\ell^{\text{off}} = \exp(-\gamma\,\ell)\,\cos\!\big(\omega\,\ell + \delta\sin(\omega\,\ell)\big), \qquad h_\ell^{\text{off}} = \frac{\tilde{h}_\ell^{\text{off}}}{\sum_{k=1}^{M} |\tilde{h}_k^{\text{off}}|}, \qquad \ell = 1, \ldots, M. \tag{23}$$

A kernel support of $M \geq 2\tau_{\text{mem}}$ suffices to capture the full tail. Typical parameter choices are $\tau_{\text{mem}} = 40$–60 bins, $Q = 2$–3, and $\delta = 0.5$–0.7.

.0.3 Evolution equations

Decompose A_{true} into diagonal and off-diagonal parts:

$$A_{\text{true}}^{\text{diag}} = \operatorname{diag}\!\big((A_{\text{true}})_{11}, \ldots, (A_{\text{true}})_{NN}\big), \qquad A_{\text{true}}^{\text{off}} = A_{\text{true}} - A_{\text{true}}^{\text{diag}}. \tag{24}$$

The history drive depending on the past M time bins is:

$$H_t = A_{\text{true}}^{\text{diag}}\,Y_{t-1} + A_{\text{true}}^{\text{off}}\left(\sum_{\ell=1}^{M} h_\ell^{\text{off}}\,Y_{t-\ell}\right). \tag{25}$$

The diagonal term uses only the most recent time bin (lag-1 self-coupling), while the off-diagonal term integrates kernel-weighted spike history over M lags. Spikes are then generated as in Model A:

$$\eta_t = H_t + B, \qquad \tilde{\eta}_t = \min(\eta_t,\ \eta_{\text{clip}}), \qquad \mu_t = \exp(\tilde{\eta}_t)\,\Delta t, \qquad Y_t \sim \text{Poisson}(\mu_t). \tag{26}$$